

Phishing Email Detection Model

A machine learning project that classifies email messages as Phishing or Safe using Python, Scikit-learn, Pandas, and TF-IDF text features.

Project status: Working prototype. The current experiment uses a small 6-email demonstration dataset, so the measured accuracy is not representative of a real-world phishing detection system. A larger, diverse dataset is required for meaningful performance evaluation.

1. Project Overview

Phishing emails are designed to trick users into revealing credentials, clicking malicious links, opening attachments, or providing sensitive information. This project demonstrates how machine learning can be used to automatically classify an email message as either:

Phishing --- suspicious or malicious-looking email content

Safe --- legitimate-looking email content

The model learns patterns from labeled email text and then predicts the class of unseen messages.

2. Objective

The main objectives of this project are:

Build a phishing-email classification model using Scikit-learn.

Train the model using labeled phishing and legitimate email messages.

Convert email text into numerical features using TF-IDF.

Train a supervised machine learning classifier.

Evaluate the model using accuracy and a classification report.

Extend the project with URL, keyword, header, and IOC-based features

in future versions.

3. Key Features

Phishing vs. Safe email classification

Text preprocessing with TF-IDF

Train/test dataset split

Machine learning classification using Multinomial Naive Bayes

Accuracy calculation

Precision, recall, and F1-score reporting

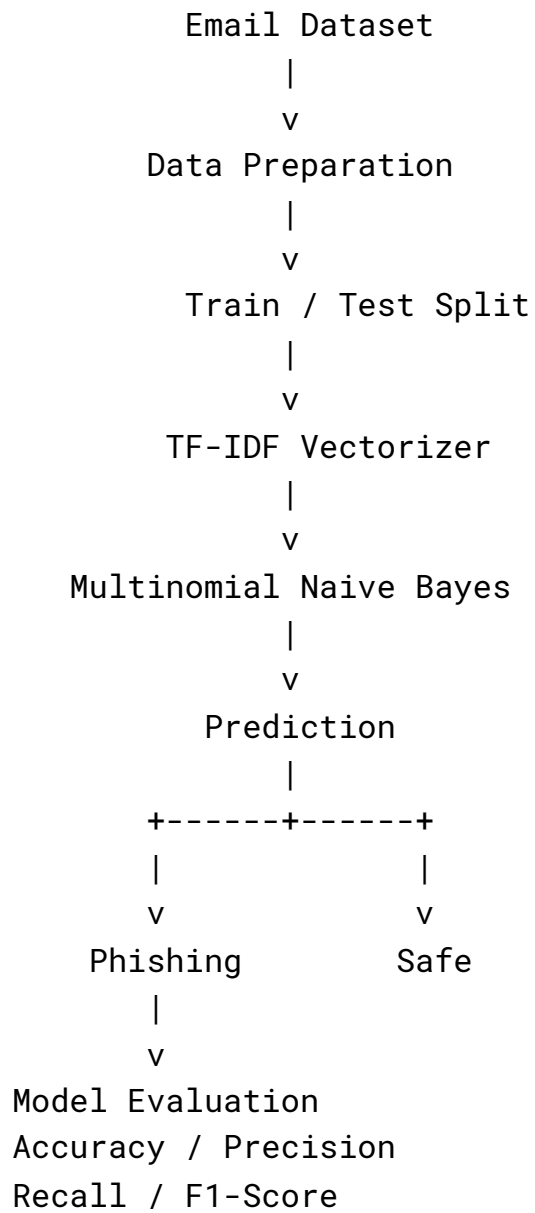
Simple Python-based implementation

Suitable as a cybersecurity + machine learning portfolio project

4. Technologies Used

Technology	Purpose
Python 3	Programming language
Pandas	Dataset handling
Scikit-learn	Machine learning
TF-IDF Vectorizer	Text feature extraction
Multinomial Naive Bayes	Classification
Git/GitHub	Version control and project hosting
Kali Linux	Development environment
VS Code / Mousepad	Code editing

5. Project Workflow



5. Dataset

The current prototype uses a small manually created demonstration

dataset containing examples such as:

Lottery/prize messages

Urgent account verification messages

Payment-related messages

Normal workplace communication

Normal account/invoice communication

The dataset contains two labels:

0 = Safe

1 = Phishing

Example:

"Congratulations! You've won a lottery. Click here to claim."

Label: 1 (Phishing)

"Meet your new colleague at the office."

Label: 0 (Safe)

Important

The current dataset contains only a few demonstration samples. Because of this, the model's evaluation score can change significantly depending on which samples are placed into the test set.

For a realistic project, the dataset should contain hundreds or thousands of diverse phishing and legitimate emails.

7. Machine Learning Method

TF-IDF

TF-IDF stands for Term Frequency-Inverse Document Frequency.

It converts email text into numerical values that a machine learning algorithm can understand.

Conceptually:

Email Text

|

v

Tokenization

|

v

Word/term frequency

|

v

TF-IDF values

|

v

Numerical feature matrix

Words that are useful for distinguishing messages receive stronger feature weights.

Multinomial Naive Bayes

The project uses:

```
from sklearn.naive_bayes import MultinomialNB
```

Multinomial Naive Bayes is commonly used for text classification because it works well with word-frequency and TF-IDF-style features.

8. Current Implementation

The core implementation performs the following steps:

```
X = df["text"]
```

```
y = df["label"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,
```

```
y,
```

```
test_size=0.2,
```

```
random_state=42
```

```
)
```

```
vectorizer = CountVectorizer()
```

```
X_train_vectorized = vectorizer.fit_transform(X_train)
```

```
X_test_vectorized = vectorizer.transform(X_test)
```

```
model = MultinomialNB()
```

```
model.fit(X_train_vectorized, y_train)
```

```
y_pred = model.predict(X_test_vectorized)
```

The implementation then calculates:

```
accuracy_score(y_test, y_pred)
```

```
classification_report(y_test, y_pred)
```

9. Evaluation Result

In the current screenshot/test run, the model produced:

Accuracy: 0.00

The classification report also showed zero precision, recall, and

F1-score for the two classes on the test set.

Why is the accuracy 0%?

This does not mean Scikit-learn or the algorithm is broken.

The main reason is the extremely small demonstration dataset. With only six total examples and a 20% test split, the test set contains only two emails. A model can easily predict both test samples incorrectly.

For example:

Total samples = 6

Training set = 4

Test set = 2

With only two test samples:

2 correct -> 100%

1 correct -> 50%

0 correct -> 0%

Therefore, this result should not be presented as the real-world accuracy of the model.

10. Confusion Matrix

For the final version, the project should generate a confusion matrix:

	Predicted	
	Safe	Phishing

Actual Safe	TN	FP
-------------	----	----

Actual Phishing	FN	TP
-----------------	----	----

Where:

TP (True Positive): Phishing email correctly detected

TN (True Negative): Safe email correctly identified

FP (False Positive): Safe email incorrectly classified as phishing

FN (False Negative): Phishing email incorrectly classified as safe

For a cybersecurity detection system, reducing false negatives is particularly important because a missed phishing email can reach the user.

11. Feature Engineering -- Planned Enhancement

A stronger version of this project should combine text features with email-specific security features.

Text Features

TF-IDF

Word frequency

Character frequency

N-grams

Suspicious keyword count

URL Features

Number of URLs

URL length

Suspicious domain patterns

IP address used as URL

HTTP vs HTTPS

Number of redirects

Suspicious TLD patterns

Email/Header Features

Sender address

Reply-To mismatch

Display-name mismatch

Number of recipients

Attachment presence

Attachment type

Header anomalies

Security Features

IOC extraction

Domain reputation

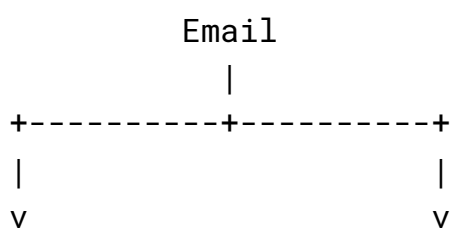
IP reputation

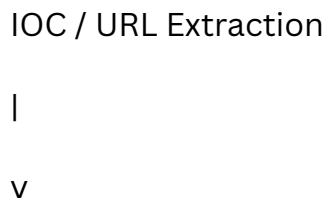
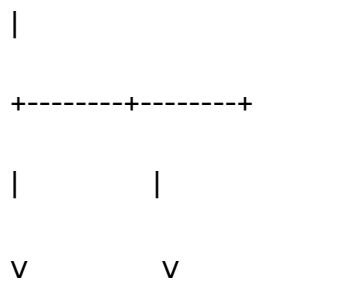
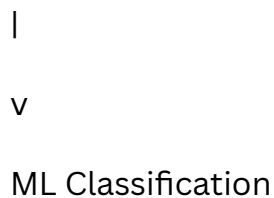
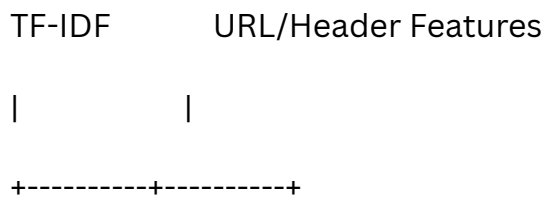
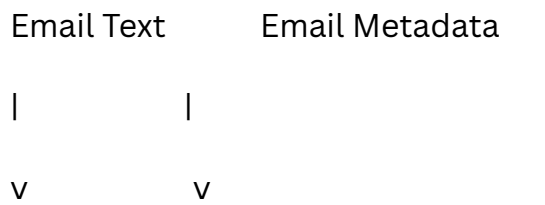
File hash

Malware indicators

Suspicious PowerShell/CMD references

12. Recommended Final Architecture





SOC Investigation

13. Installation

Clone the repository:

```
git clone https://github.com//phishing-email-detector.git
```

```
cd phishing-email-detector
```

Create a virtual environment:

```
python3 -m venv venv
```

```
source venv/bin/activate
```

Install dependencies:

```
python -m pip install --upgrade pip
```

```
python -m pip install pandas scikit-learn matplotlib seaborn joblib streamlit
```

Verify Scikit-learn:

```
python -c "import sklearn; print(sklearn.version)"
```

14. Run the Project

Run the training script:

```
python e-mail.py
```

For a larger final implementation, a recommended structure is:

```
phishing-email-detector/
```

```
|
```

```
|—— dataset/
```

```
|  |—— emails.csv
```

```
|
```

```
|—— models/
```

```
|  |—— phishing_model.pkl
```

```
|  |—— tfidf_vectorizer.pkl
```

```
|
```

```
|—— results/
```

```
|  |—— confusion_matrix.png
```

```
|
```

```
|—— src/
```

| |—— features.py

| |—— train.py

| |—— predict.py

|

|—— app.py

|—— requirements.txt

|—— README.md

|—— .gitignore

15. Future Improvements

The following improvements can make the project significantly stronger:

Use a large real-world phishing/legitimate email dataset.

Add proper text cleaning and preprocessing.

Use TF-IDF with unigrams and bigrams.

Add URL extraction and URL-based features.

Add suspicious keyword analysis.

Add sender and Reply-To analysis.

Add email-header analysis.

Compare multiple algorithms:

Logistic Regression

Multinomial Naive Bayes

Linear SVM

Random Forest

Use cross-validation.

Handle class imbalance.

Generate confusion matrices and ROC/PR curves.

Save the trained model using Joblib.

Build a Streamlit web interface.

Add IOC extraction for URLs, domains, IPs, and hashes.

Map suspicious behavior to MITRE ATT&CK techniques.

Integrate the output into a SOC-style alert workflow.

16. Cybersecurity Use Case

This project demonstrates how machine learning can support a SOC analyst.

Example workflow:

Incoming Email

|

v

Phishing Detection Model

|

v

Suspicious Email Detected

|

v

Extract IOC

(URL / Domain / IP / Hash)

|

v

IOC Enrichment

|

v

SOC Analyst Investigation

|

Contain / Block / Report

The model is intended as a detection aid, not as a replacement for analyst investigation.

17. Security Note

Do not click or open links and attachments from suspicious emails while testing the project.

For safe analysis, treat URLs as strings and use isolated/sandboxed environments when deeper malware analysis is required.

18. Project Outcome

This project demonstrates the fundamentals of:

Cybersecurity

Phishing detection

Machine learning

Natural language processing

Feature extraction

Classification

Security alert analysis

SOC investigation concepts

The current implementation is a proof-of-concept. The next major step is to train and evaluate it on a sufficiently large and representative dataset before making performance claims.

19. Resume Project Description

You can use the following description on your resume:

Phishing Email Detection using Machine Learning | Python,
Scikit-learn

Developed a machine-learning-based phishing email classification prototype using Python and Scikit-learn.

Applied text vectorization and supervised classification to distinguish phishing and legitimate email messages.

Implemented train/test splitting and model evaluation using accuracy, precision, recall, and F1-score.

Designed the project for future integration of URL analysis, IOC extraction, email-header analysis, and SOC investigation workflows.